

Stellar-based integrated health records and drug supply chain ▾

None of the major solutions identified in the prior art investigation are built on Stellar. Those systems are predominantly on **Ethereum**, **Hyperledger Fabric**, or **Hedera**.

However, I have found several separate projects that use the Stellar network for both healthcare records and drug supply chains.

📄 Stellar-Based Health Records Solutions

* **Healthv-Stellar**: An **open-source health records platform**

Show more

Crafting a high-quality, specific integration plan document >

I'll put together a comprehensive integration plan. Let me first check whether there's a markdown skill to follow for formatting conventions.

Synthesized integrated Stellar solution bridging healthcare records and drug supply chains >

No dedicated markdown skill exists, so I'll write the plan directly. Here's my thinking before I draft it: the real gap your research surfaced is that nobody has fused the two domains, and the natural seam between them is the **prescription** — it's simultaneously a health-record event *and* a demand signal in the supply chain. That single insight is what the whole architecture should pivot around, so I've built the plan to treat the prescription as the bridge primitive.

Integration plan markdown document combining health records and drug supply chain on Stellar >

continue, I see the plan in markdown file but it looks good as it is, is there anything else you need to add?

Presented file >

Glad it reads well. As a *strategy and architecture* document it's complete — it has the thesis, the differentiator, the modules, the workflow, compliance, roadmap, and risks. So nothing is *missing* in the sense that it stands on its own.

That said, if you're going to take this to a team or a funder, there are four genuine gaps I deliberately left out to keep it focused, and you may want one or more filled in:

1. **Resourcing & timeline.** Right now the roadmap has phases but no team size, rough budget, or duration per phase. A reviewer's first question is usually "how long, how many people, how much."
2. **Network governance & operations.** The plan assumes credentials get issued by "recognized authorities" and contracts get upgraded — but it doesn't say *who runs the network*, who controls credential issuance and revocation, or how contract upgrades are governed. For a multi-party healthcare network this is often the make-or-break question.
3. **Soroban storage-rent / TTL strategy.** I flagged this as a risk but didn't solve it. With prescriptions, consent grants, and per-unit custody hops accumulating, you need a concrete policy for what stays in live state, what gets archived, and who pays the rent. It's the one place the "high-frequency events are cheap" argument could quietly break.
4. **Business model — who pays.** I mention anchors and copays in passing, but there's no stated answer to who funds the system (transaction fees, SaaS to pharmacies, payer subsidy). The incentive token rewards good behavior but doesn't explain where the value comes from.

My honest take: of these, **governance (#2)** and **storage strategy (#3)** are the two I'd add before showing this to anyone technical, because they're the questions a skeptic reaches for first and the current draft has visible seams there. The other two are easy to defer.

Want me to add a governance section and a storage-rent strategy section, or expand all four?



Stellar integrated health supply plan
Document · MD



Download

clinical-history-
tiers.md

378 lines

MD

ok flesh out the rest of the document then, adding those missing points. Minus the resourcing and timeline, this is an MVP so it's just me and the rest can just be mention as a proposed plan in the future. This is focused on the Stellar network but the idea is to have a decentralized network function here where a smart contract is deployed that verifies a client with a authentic copy of the clinical history and the doctor/institution requesting access and doing the audit / act as bridge for providing access to the requested data. The tiers of data are in question, addressed in the markdown file attached.

Orchestrating comprehensive document rewrite via integrated architecture >

The attached tier spec slots in perfectly — it's exactly the data model that the "smart contract as access broker" needs, and Tier 3 is also where the prescription write-back lands. Let me fold all of it in: the access-broker architecture you described (verify authentic copy → verify requester → audit → bridge access), the three tiers, plus governance, Soroban storage strategy, and a future-tagged business model. I'll drop resourcing/timeline and reframe the roadmap as solo-MVP scope.

Let me look at the current file so I integrate rather than clobber.

Architected comprehensive document rewrite integrating Stellar network framework >

The structure is as I left it. Given the scope of additions and the renumbering they require, I'll rewrite the file cleanly — preserving all the strong existing content and weaving the new material into a coherent flow.

Ran a command, read a file >

Done. The plan now runs 15 sections and folds in everything you specified. Here's what changed and why:

Your access-broker architecture is now the second pillar of the whole design (alongside the prescription bridge). I made the contract's four jobs explicit and concrete — verify authenticity (commitment check), verify the requester (credential + grant), audit (events), bridge access (capability release) — and added a sequence diagram showing a read ending in the requester verifying the retrieved blob against the on-chain commitment. That last step is your "authentic copy" guarantee, made mechanical.

The three tiers got a dedicated section (§6) with a full comparison table mapping each tier to its broker check, audit behavior, and on-chain/off-chain split. The one detail I want to flag because it's a genuine design win: I modeled **Tier-2 break-glass grants as Soroban Temporary storage**, so expiry literally is revocation — the emergency window evaporates on its own, no cleanup transaction, no forgotten open door. I also added an emergency break-glass workflow diagram (§7.2) covering both the network-down and network-up paths.

The two gaps you blessed are in, framed honestly:

- **Governance (§10)** — stated plainly that the *ledger* is decentralized day one but the *app layer* starts as a single admin key, with every centralized MVP choice paired to a named `[future]` successor (multi-sig/governance, federated issuers).
- **Storage/TTL (§9)** — concrete policy: Instance/Persistent/Temporary used deliberately, audit emitted as **events** (so the trail grows without rent), cold data allowed to archive and restore-on-demand, and gasless UX via fee-bump + sponsored reserves.

Business model (§11) is included but marked `[future]` and kept short, with the rule "patient never pays, clinical choices never monetized."

Resourcing and timeline are gone. The roadmap (§13) is now solo-MVP scope ordered by *capability*, and follows your companion note's Tier 3 → Tier 2 → Tier 1 build order, with `[future]` tags on anything multi-org.

One thing worth your attention before you build: §8 and §14 both call out that the **Tier-2 firewall** (keeping supply-chain/reservation history out of emergency bundles) and the **linkage-attack surface** (public reservations leaking who's sick) are the two places the privacy model is most likely to spring a leak. If you want, I can expand either into a short threat-model appendix with concrete mitigations — that'd be the natural Phase 0 artifact.

Stellar integrated health supply plan

Document · MD

Download

As an adversary agent tasked with identifying vulnerabilities in this architecture, I am looking for the "seams"—the places where the private and public gr...

PASTED

Address these. For example, the social engineering part, we currently have a situation where the the doctors themselves fly blind when dealing with a patient so that's a net negative than some rogue actor playing hacker and stealing info pretending to be a doctor. We have to trust that institutions that are want to have read access inside the system are onboarding correctly and have to verify identity before entering into smart contract relations with the client. Second, think outside the box, I can list a huge list of terms regarding Zero knowledge proofs, merkle trees, attestation mechanism, think of what can solve these issues with detail

Synthesized cryptographic primitives addressing six architectural vulnerabilities >

This is a genuinely good red-team pass — and your steer on #5 is the right instinct, so let me start there because it reframes how aggressively to harden everything else.

The break-glass risk calculus — you're right, and here's the principle. The dominant, recurring, real-world harm today is clinicians flying blind in an emergency. That's a *systemic* failure that kills people regularly. A stolen-credential data theft is *rarer, bounded, attributable, and revocable*. Optimizing the architecture against the rare attacker by adding friction to the legitimate emergency read would be trading a common fatal failure for a rare bounded one — a bad trade. So the correct control isn't friction at the read path; it's **identity assurance at onboarding**. Issuers must verify institutional *and* individual identity before any credential enters a smart-contract relationship. The contract trusts a responder because the institution that vouched for them is itself verified and slashable/revocable. That's where the trust is anchored, exactly as you said.

But we can still kill the *mass* version of the attack cheaply, and that brings me to the one idea that solves both #5 and #6 at once:

The unifying primitive: proof of patient physical presence. Bind every privileged access to cryptographic evidence that the patient — or their physical token — is *right there*. The patient carries an NFC card/bracelet (passive signer) and a wallet key (active signer). A break-glass request must include a fresh nonce signed by tapping the patient's card; a dispensation must include the patient's co-signature. The effect is that an action can only ever target *the patient in front of you*, never an arbitrary row in the database. A stolen ambulance tablet, far from any patient, can break glass on *nobody* — it has no patient token to tap. An unconscious patient still has their bracelet, so the legitimate case works passively. This converts "scrape high-profile patients from the database" into "you can only read the one patient whose wrist you're touching," which is the security boundary you actually want for emergency care. The residual (someone steals both the tablet *and* a specific patient's bracelet) is bounded to one patient and noticed — a completely different threat class from a database scrape.

Now each hole with the specific primitive that closes it:

1. Metadata breadcrumb (the most serious, you're right). The fix is to stop putting the patient ↔ drug edge on the public graph at all, and to promote unlinkability from

[future] to MVP-core. Two layers:

- *One-time stealth addresses per prescription.* Derive a fresh, unlinkable address for each prescription (HD-derivation the patient can recognize but outsiders can't correlate). This breaks "this pseudonym keeps reserving oncology meds" — there's no persistent pseudonym to build a behavioral map from.
- *Commit to the drug class, don't publish it.* The reservation records a Pedersen

commitment to the drug within an anonymity set, plus a proof that "a valid, credentialed, unused prescription exists whose class matches the batch being reserved." The supply chain verifies the reservation is *legitimate* without learning the patient or the exact molecule. The MVP-cheap version is the stealth address + class-commitment with a Merkle-inclusion proof; the hardened version replaces it with a ZK reservation proof. The public graph still gets what it legitimately needs — provenance of the *unit* — but the patient linkage is no longer a public edge.

2. Oracle integrity. You're right that crypto can't verify physical reality, so the honest goal is to make physical fraud *attributable and expensive*, not impossible. The move is to kill the single-source oracle:

- *Multi-party attested handoffs.* Each custody hop becomes a sender-claim + receiver-verify co-signature. The distributor's "all good" alone is worthless; the receiving pharmacy independently reads and co-signs. Fraud now requires *collusion* across custody parties, not one bad actor with an insulated sleeve.
- *Secure-element sensor signing + irreversible chemical indicators.* Sensors sign from a hardware key (stops in-transit data tampering), and a one-way chemical temperature indicator on the package gives a *physical* cross-check the receiver attests to. The insulated-sleeve trick defeats a lone digital sensor but not a receiver who looks at a tripped chemical strip and refuses to co-sign.
- *Statistical anomaly detection + slashing.* A too-perfect flat temperature line is itself suspicious; flag it. Pair with the [future] stake/slashing so a caught spoofer loses money. The blockchain's contribution is turning anonymous untraceable fraud into a signed claim by an identified, slashable party.

3. Tier-1 ghost audit. The adversary conflated Tiers 1 and 2 — and separating them is most of the fix:

- *Tier 2 is never offline-readable.* Only Tier 1 is, and Tier 1 is **non-stigmatizing by design** (allergies, "has a pacemaker," critical meds — facts whose leak is survivable). So the offline path can never expose the sensitive bundle. Break-glass to Tier 2 always requires network, hence always produces an immediate audit.
- *Finite signed offline budget.* A responder credential carries only N offline-read capabilities that expire; the device *must* resync to get more. A permanently-offline stolen reader can scrape N cards, not the database — blast radius capped.
- *Hardware-sealed local hash-chain.* Each offline read appends to a tamper-evident chain in the secure element; the stolen device physically *carries forensic evidence* of every read it performed, recoverable on seizure.
- *Patient as second auditor.* The card emits a rolling code per read, so when the

patient's own device later syncs it can detect "my card was read three times" even if the responder never posted anything. The audit stops being a pinky-promise because there are two independent witnesses.

4. Denial of history (rent). The rule is: *the emergency-critical path must never depend on live, rent-paid state.*

- *Self-verifying cards.* The Tier-1 card carries a signed payload plus the issuer's certificate. Verifying it needs the issuer's public key (stable, cached) — not a live ledger lookup. So a crisis read works even if every Persistent entry has lapsed.
- *Merkle-rooted critical commitments.* Keep one tiny always-alive root entry covering all of a patient's emergency-critical commitments, with individual records proving inclusion via Merkle path. You're paying rent on *one small entry*, not N, so keeping the critical subset alive is nearly free, and a keeper service (anyone can bump Soroban TTL) auto-renews it with a wide grace margin and alerting. Bulk cold history is allowed to archive and is restorable on demand — but the crisis path never touches it.

5. Break-glass escalation. Covered by the reframing above plus defense-in-depth: proof-of-presence (binds to the physical patient), an institutional co-signature alongside the responder credential (the institution you trust to onboard correctly is the live gatekeeper and can revoke instantly on a theft report), rate-limit/anomaly-gating, a patient-defined emergency network, and graduated escalation (a vital safety subset on single-responder break-glass vs. a fuller bundle needing a receiving-clinician co-sign).

6. Bridge race / pharmacy key.

- *Escrow with TTL.* The reservation is a contract-held escrow on a Temporary entry. A lock-without-dispense (your DoS) auto-reverts on expiry and returns inventory — no permanent lock. DISPENSE *atomically consumes* the reservation, so there's no window for a double-spend.
- *Patient co-sign kills ghost dispensation.* Dispensing to a ghost pseudonym fails because DISPENSED requires the patient's proof-of-presence co-signature — a ghost can't sign. A compromised pharmacy key alone can't drain inventory to fake patients.
- *Multisig + rate-limit even in MVP.* I'd push back on the single pharmacy key: make the pharmacy identity a 2-of-3 and rate-limit dispensations per period, with a reconciliation freeze when on-chain dispensed counts diverge from physical audits. Even pre-token, that caps a compromised-key burst.

On Soroban feasibility so this isn't hand-waving: Merkle-inclusion and hash checks are

cheap native ops, Ed25519 co-signatures (the proof-of-presence) are native, and pairing-based proof verification (BLS12-381) is feasible on-chain for the ZK paths — proof *generation* is client-side and is the real cost, which is exactly why I'd stage the cheap stealth-address-plus-commitment approach first and the full ZK reservation proof later.

Let me fold the durable decisions into the plan.

Viewed 2 files, edited 2 files, read a file >

I've added §16 (Adversarial hardening) capturing all six mitigations, the proof-of-presence primitive, and your break-glass risk calculus, and strengthened the §8 break-glass bullet to make Tier-2 network-only with proof-of-presence + institutional co-signature.

Two honest caveats I want to leave you with rather than bury:

The proof-of-presence primitive is the strongest idea here, but it has one genuine failure mode worth designing around: the patient who arrives at the ER *without* their card or bracelet (lost it, house fire, mugging). You can't let "no token = no emergency access," or you've reintroduced the flying-blind problem the whole tier system exists to prevent. So Tier 1 needs a tokenless fallback — likely a biometric or a clinician-attested identity lookup that opens a *narrower, more heavily-audited* path than a card tap. That's a real Phase-0 design decision, not a detail.

And on #1 (metadata): stealth addresses plus a class-commitment genuinely break the *persistent behavioral map*, but they don't fully erase timing-and-location correlation on their own — a determined observer watching one specific pharmacy can still infer "someone filled an oncology script here at 2pm." Fully closing that needs the k-anonymity batching to actually have a population to hide in, which is an adoption problem, not a crypto one. Worth being clear-eyed that early, low-volume MVP usage is *inherently* less private than the steady state, so I wouldn't make strong anonymity promises to pilot patients.

I took you up on the offer implicitly — if you want, the natural next step is to go down to the Soroban function level for the access broker (the `request_access` → verify-credential → check-grant → emit-audit → release-capability path, plus the proof-of-presence co-sign check), since that's the contract where the most subtle bugs will live.



Stellar integrated health supply plan
Document · MD



Download

yea I agree, let's say that we are designing around something that is the responsibility of the patient. If we veer into biometric identification, that would be on the onus of a big nation state to keep and plug in and we still want to respect the privacy and rights of an individual to belong in society without needing to wear the bracelet if they want to, they just acknowledge that being provided care in a rare emergency would be a big hassle to the doctors and raises the probability that adequate care would

Show more

Architected Soroban contract design with security hole analysis >

Locking your philosophy in as hard design constraints before I touch the contract, because they actually change the function signatures:

- **The token is opt-in and the patient's responsibility.** No biometrics, no nation-state identity registry. The contract supports two emergency paths: a *presence path* for patients who registered a card/bracelet, and a *tokenless fallback* for those who didn't — deliberately narrower (vital subset only), dual-co-signed by institution + a second clinician, and more heavily audited. Opting out is a right; the cost is that the fallback is a hassle and lowers the odds of fast, complete care without ever *denying* it.
- **Residual metadata leak is accepted as "public-place equivalent."** So this turn I'm not re-litigating the anonymity-set question; the broker just has to not make it worse than the real-world baseline.

Now the contract. The single most important thing — and it's exactly the kind of hole your instinct predicted, invisible at the strategy level and lethal at the architecture level — is the **read-only simulation scrape**:

Soroban clients *preflight* every call by simulating it to get the return value and footprint. Simulation runs the contract and hands back the return value **without committing any state or events**. So if `request_access` returned the decryption key in its result, an attacker would simulate the call, read the secret straight out of the simulation response, and *never submit the transaction* — no grant committed, no audit event, zero on-chain trace. The "tamper-evident audit" would be bypassed by a call that never happened on-chain.

The defense has to be structural: the contract **never returns secret material**. It returns only a non-secret capability (locator + commitment + grant_id), writes the grant, and emits the audit event. The actual key is released by the off-chain KMS, and the KMS releases *only when it can confirm the grant is committed in ledger state* — which a simulation never produces. "Audit-first, return-a-pointer-not-a-key, KMS-checks-committed-state" is load-bearing, not stylistic.

Two more in the same family: never use Soroban storage **TTL as the authorization boundary** (anyone can `extend_ttl`, and it's ledger-granular — always check `now < grant.expires_at` in code), and never let the capability be a **bearer token** (the KMS must re-derive authorization from *current* committed grant state every time, or revocation silently breaks).

I've written the full broker design with the Rust skeleton, the presence-proof verification, the tokenless fallback, and a complete annotated bug catalog.

Created a file, read a file >

The design doc is up. The structure is hole-first: §5 is a ten-row bug catalog (A-J) and every function in §3 is annotated with which hole it's guarding. The Rust is an illustrative skeleton, not compiled — I flagged that at the top.

The three things I'd actually worry about in code review, in priority order:

Hole A (simulation scrape) is the one that's easy to reintroduce by accident. Any well-meaning refactor that makes `request_access` "convenient" by returning the wrapped key directly reopens it instantly. The invariant to enforce in review forever: *the contract's return value must be safe to publish*. If you can't tweet the return value, it doesn't belong in the return value. The §6 adversarial preflight test exists specifically to catch a regression here.

Hole C (bearer capability) is really a contract you can't see from inside the contract — it lives in the KMS ↔ broker boundary. The broker code can be perfect and you can still be owned if the KMS treats the capability as a ticket instead of re-checking committed state every release. So that's less a Soroban question than a "who builds the KMS and do they understand the threat model" question. Worth writing the KMS release check as its own spec'd artifact rather than leaving it implicit.

Hole B (TTL ≠ expiry) is the one most likely to bite a solo dev because the temporary-storage auto-delete *looks* like it's doing the security work, so the `now >= expires_at` check feels redundant and gets dropped. It isn't redundant — `extend_ttl` is permissionless. I'd put a comment that loud right on the check.

One genuine gap I left open in §7 that you'll have to decide before this is real: the **KMS is itself a re-centralization of the secret layer** — a single KMS that can re-wrap any key is a single point of compromise that sits *outside* everything we just hardened. Threshold/MPC re-encryption across independent holders is the honest answer, but it's heavy, so for the MVP it's probably a documented single-KMS trust assumption sitting next to the single-admin-key one. I didn't want to paper over it.

If you want to keep going down the stack, the natural next target is the **KMS release-check spec** (the other half of Hole A/C), or the `prescription` contract's `RESERVED → DISPENSED` atomicity with the patient co-sign — that's where the supply-chain double-spend and ghost-dispense holes from the red-team live.



Access broker contract design
Document · MD



Download



Stellar integrated health supply plan
Document · MD



Download

[Download all](#)

As your adversary agent, I've reviewed your **Hardening Update (§16)**. You've attempted to plug the holes with the **Proof-of-Presence** co-signa...

[PASTED](#)

I'm assuming that stellar itself doesn't have a KMS dApp it supports and has been audited? Here are list of decentralized KMS systems that might fit the bill. Yes, decentralized Key Management Systems (KMS) are a growing field, primarily driven by the need to eliminate the "single point of failure" found in traditional cloud providers (like AWS KMS or Google Cloud KMS).

Instead of one company holding your master key, these systems use

[Show more](#)

Distinguishing transaction-signing management from decentralized data-KMS architecture >

Before I answer the KMS question I want to verify the current state of a couple of things rather than assert from memory, since "does X support Stellar" is exactly the kind of fact that shifts.

Validated mental model, triaged security concerns, mapped decentralized KMS solutions >

Let me take the mental model first, because your instinct on #2 is right and it actually reframes the whole adversary report.

Your mental model is correct — that is what's proposed. Two addresses (requester + patient owner), an access request scoped by ACLs, the patient signs to agree, the contract opens a gate by returning a pointer + metadata, and the actual bytes move over HTTPS between the requester and wherever the data lives. That's the access-broker exactly. The one piece your description compresses is *what the HTTPS fetch actually returns*: it returns **ciphertext**. The locator points at an encrypted blob the requester still can't read. So the gate is really two gates — (1) the contract authorizes and hands over the locator (your model), and (2) something releases the *decryption key* to the requester. In the design so far that "something" was a KMS, and that's the single most important place your KMS research plugs in. Hold that thought.

Also worth naming: your description is precisely the *normal Tier-3 path* — patient conscious, signs the grant. The break-glass and tokenless flows are just the variations for when the patient *can't* sign. So you've internalized the core correctly; the emergency machinery is the exception layer on top.

Now #2 — and I'm going to disagree with your triage. You said #2 is the one that might land. It's actually the *weakest* of the five, because it attacks something that was never proposed. Stealth addresses were introduced in §16 for *one narrow job*: stop outside observers from building a behavioral map out of the **public prescription/reservation footprint** (the supply-chain graph). They were never the identity model for the clinical record. The patient's clinical history is anchored to a **stable** patient key and indexed by an off-chain **manifest** — that's the source of truth a clinician reads. A clinician with a Tier-3 grant (or an emergency bundle) navigates the manifest; they never reconstruct medication history by crawling public chain activity. So the "siloeed care / 90% missing history" scenario only happens if you naively tried to *rebuild the record from the public ledger*, which no sane design does. The adversary conflated "the patient's public reservation footprint" with "the patient's clinical identity." Separate those two and #2 dissolves. The residual — keeping the private map of *which stealth reservation belongs to which patient* — lives in the patient's own record and is recoverable (which is, again, a KMS/key-recovery question, not a fragmentation problem).

So here's the honest re-ranking of the report:

#4 (co-signature collusion) is the strongest criticism, and it genuinely lands. The adversary is exactly right: co-signatures only secure anything when the parties have *opposing* interests, and in a drug supply chain everyone shares an interest in looking green — the distributor doesn't want to eat the batch, the pharmacy doesn't want to lose the sale, both are happy to sign "healthy." My "multi-party attested bundle"

lose the sale, both are happy to sign neatly. My multi-party attested handoff quietly assumed adversarial custody parties, which is wrong. Signatures can't fix this; you need a party with *genuinely opposed* incentives in the loop. Three real options: (a) push the binding check downstream to where liability flips the incentive — a pharmacy that dispenses heat-damaged drugs carries malpractice/regulatory liability that exceeds the sale value, so its co-sign stops being free; (b) put the *patient/clinician* (harmed by bad drugs, no stake in the sale) or an *insurer/regulator* (pays for failures) as the attesting party of record; (c) hardware the colluders don't control — an irreversible chemical indicator and a secure-element sensor whose key isn't held by either custody party, plus random independent spot-audits with slashing sized so expected fraud cost > batch value. The blockchain's honest contribution here isn't prevention; it's making the false "green" claim **attributable** so that when a patient is harmed, the signed lie and its signers are on permanent record and liability attaches. I'd concede this one in the plan and rebuild the cold-chain trust around an opposing-interest attester.

#1 (NFC relay) is real but bounded in this threat model. Relay attacks against proximity protocols are a known, legitimate class — signal ≠ location is correct. But the *payoff* of a successful relay here is: a Tier-2 bundle, for one physically-near patient, with a valid responder credential, that produces an on-chain audit event and a patient notification. That's a targeted, audited, credential-gated read — not silent mass exfiltration — and it sits inside the §16.3 risk envelope you already accepted. Two things tighten it without hurting the emergency case: for the **dispensation** path (patient conscious) require an *active* co-sign on the patient's own device rather than a passive tap — a relayed passive nonce can't produce an active confirmation, so relay dies there; for the **emergency** path (patient unconscious, can't act) accept the bounded residual, since the alternative is the patient who doesn't get care. So #1 is "real, mitigated where the patient is awake, accepted-and-audited where they aren't."

#3 (hash chain = forensics) is a fair hit on one layer, but misreads the stack. The adversary's right that a destroyable device's local log isn't prevention — but the hash chain was explicitly the *forensic* layer, not the *preventive* one. The prevention was: Tier-2 is never offline (so the offline path only ever exposes deliberately non-stigmatizing Tier-1), and the offline read budget is a *finite signed capability* — the device can perform N reads and then is **cryptographically unable** to do more without resyncing. So "extract data and smash the reader" nets the attacker a bounded number of *non-stigmatizing* cards, not a database. The deeper truth the adversary is circling, and I'll just concede it plainly: offline access *cannot* produce on-chain audit at read time — that's intrinsic to "offline." This is a real CAP-style choice. You're choosing availability (a paramedic acting on a dead network) over auditability, and bounding the cost by making the offline surface small, non-sensitive, and budget-capped. That's defensible, but it should be written as an explicit accepted tradeoff, not dressed up as solved.

solved.

#5 (nonce UX DoS) — and this is your "bricked token" question. The adversary is right that you must never let "the NFC won't scan" be the reason a patient doesn't get a drug. The answer is the principle that proof-of-presence is **preferred-when-available, never a precondition**. Tap fails — damaged antenna, dead reader, no bracelet, patient opted out entirely — and the system falls through to the **tokenless fallback** that's already in the broker (`authorize_emergency`'s `None` branch): institution + second-clinician dual co-sign, vital-subset only, distinct heavily-audited topic. Care proceeds. The reason that's not "a massive backdoor" is that the fallback is deliberately *narrower and noisier*, not *easier* — it gets you the vital subset under two credentials and a loud audit trail, not the full record under a quiet one. So the failure mode of the token is degraded-but-functional access, never denial.

On the KMS — you're right, and it closes two of my own open holes. I confirmed Stellar has no first-party audited data-KMS: key management in the Stellar world is wallet/MPC-vendor territory (e.g. Blockdaemon's Builder Vault offers MPC signing that covers Stellar and Soroban, but that's enterprise transaction-signing, not a decentralized service that releases *decryption* keys against on-chain conditions). So bringing an external decentralized KMS is the right instinct, and it directly retires the two things I flagged last turn: the **single-KMS re-centralization** (the §7 open question / Hole C) and the **bricked-token / lost-key** problem. Blockdaemon

But there's one caveat that applies to *all four* of your candidates and it's the thing to evaluate before committing: **none of them natively read Stellar/Soroban contract state**. Mapping them onto the design:

- **Threshold Network (PRE)** is the cleanest *conceptual* fit for the data-key layer — proxy re-encryption across independent nodes that release re-encryption fragments only when a threshold participates is *exactly* the "release the key against a committed grant" gate, decentralized. It's the textbook replacement for the centralized KMS in Hole C.
- **Lit Protocol** is the most *programmable* fit — threshold decryption gated by conditions. But its native access-control conditions read state from most EVM chains, Cosmos, and Solana — **Stellar isn't on that list**. The realistic path is a **Lit Action** (their in-network JavaScript) that fetches a Stellar RPC endpoint to check whether the grant is committed/unrevoked/unexpired, then conditionally decrypts — Lit Actions are designed to transmit information from a blockchain to a server and return a signed proof, so it's feasible. The cost: you're now trusting that RPC fetch inside the Lit Action — a *new* trust seam exactly at the join you spent all this effort hardening. Whatever you pick, that Stellar-state-verification bridge is

where the next adversary will aim. [Litprotocol](#) [Lit-protocol](#)

- **Web3Auth** solves a *different* hole — it's for the **patient's** key, not the data-encryption key. It's chain-agnostic and natively supports the Ed25519 curve via threshold (FROST) signing that Stellar uses, splitting the key into device + social + backup factors so a lost phone or dead bracelet isn't a lost medical history. That's your bricked-token recovery and your "lost key = lost record" answer, sitting right next to (not replacing) the data-KMS. [Web3Auth Blog](#)
- **Zama (FHE)** is the wrong tool here — it's for *computing on encrypted data on-chain*, which you don't need since the data stays off-chain. Skip it; it's heavy and aimed at a problem you designed around.

So the shape is: **Threshold PRE (or Lit) for the data-key gate, Web3Auth-style MPC for patient key recovery, and a Stellar-state-verification bridge between them that is itself the thing to red-team next.**

Want me to (a) fold the #4 collusion rebuild and the #1/#3/#5 concessions into §16 of the plan, and (b) write the KMS layer into the broker design — specifically the Threshold-vs-Lit decision and the spec for that Stellar-state bridge, since that bridge is now the sharpest remaining seam?

right and like even then the emergency path , if the person is awake and normal can be rejected within a certain timeframe , as in the attacker needs to be close for at least 30 seconds and during those 30 seconds, it identifies to the user it can prevent this deluge of information being shared. but of course , this scopes the doctor to waiting 30 seconds before actually providing care which i believe something much better than the current actual reality of the situation of no clinical records at all.

Show more

Architected veto-window mechanism and formalized Lit-Stellar KMS integration >

Two things here — the veto-window idea (which is genuinely good and I want to push on slightly) and the actual deliverable.

The 30-second veto window is the right shape, and it's better than I think you're giving it credit for. What you've described is an *opt-out within a window* rather than opt-in, and its elegance is that it auto-distinguishes the two cases without the patient doing anything in the legitimate one. Unconscious patient: no veto arrives, data flows after the window — exactly what you want. Conscious person being covertly leeched in a waiting room: they get a prompt and kill it. The cost is a bounded delay, and you're right that a bounded delay beats the status quo of zero records.

One refinement, because 30 seconds *can* matter in the genuine worst cases (anaphylaxis, arrest): **graduate the delay by criticality, not a flat window.** The tiniest, most time-critical, least-stigmatizing micro-subset — severe allergies, anticoagulants, implanted devices — releases *instantly* (these are the "if you know nothing else, know this" facts, and their disclosure is the least harmful), while the fuller Tier-1 payload and the pointer to Tier-2 carry the veto window. So you never make a doctor wait 30s to learn the patient is on warfarin, but you do make the marginal data wait. That keeps the life-saving immediacy where it actually saves lives.

Honest residual to write down: the veto depends on the patient being *notified and able to act* in the window. Dead phone, or an attacker who jams notifications while relaying, defeats it — but that's now a high-effort, targeted, on-stigmatizing-data-only attack against one physically-present person, which is inside the envelope you've already accepted. Worth stating, not worth over-engineering.

On the deliverable: Lit-for-MVP with stubs and a formal interface is exactly the right call, because it lets you commit to the *contract* now and defer the *implementation*. I picked TLA+ as the formal method, because the property you actually care about — "a key is never released except against a committed, unrevoked, un-vetoed, in-window grant" — is a safety invariant over a small protocol, which is TLA+'s sweet spot and is model-checkable in TLC. The nice payoff: your 30-second idea becomes a machine-checkable theorem (**VetoSafety** : a vetoed grant is *never* released, which holds by construction because veto can only fire before reveal-time and release requires reveal-time has passed), and the simulation-scrape defense becomes another (**CommitSafety** : no release without committed on-chain state).

I've written the full spec — TLA+ model, the design-by-contract interface both the stub and the real Lit gate must satisfy, the Lit Action that bridges to Stellar state, and the MVP stub boundary.

Created a file, read a file >

The spec is up. The structure deliberately makes your veto idea and the simulation defense *the same kind of object* — both are TLA+ safety invariants over one small protocol, so they're checkable in TLC rather than argued in prose:

- `VetoSafety` is your 30-second window as a theorem: a vetoed grant is *never* released, and it holds by construction because veto can only fire while `clock < revealAt` and release requires `clock ≥ revealAt`, with `vetoed` monotonic — so a vetoed grant can never satisfy the release guard, ever.
- `CommitSafety` is the simulation-scrape defense: no release without *committed* on-chain state, which a preflight never produces.
- The criticality-graduated window is parameterized as `Window(class)`, so the warfarin/allergy micro-subset is `Window = 0` while the rest is `30s`, and both invariants still hold per class.

The one design rule I'd burn into the MVP is in §6: **stub the decentralization, never stub the predicate**. Single-server KMS is fine for now (it's the same documented trust compromise as the single admin key), but the release predicate, the veto, and revocation are *real from day one* because the safety properties depend on them. That keeps the stub and the eventual `LitGate` as a clean refinement — same interface, same boolean, only "who holds the keying material" changes.

And the honest pointer for next time someone red-teams this: the seam is no longer your Soroban logic or your KMS — it's the **RPC read inside the Lit Action** (§4). If an attacker can make enough Lit nodes believe a lie about Stellar state (a forged or stale RPC response), the predicate passes on false premises. The mitigation is N-of-M nodes each reading *independent* RPCs and agreeing, plus committed-only reads — but that's the surface to specify carefully before this is real, and it interacts with the TTL point (an archived grant entry reads as "absent" → deny, so a legitimate emergency read can fail for a *storage-rent* reason, which is a correctness bug, not just hygiene).

Two natural next steps if you want them: I can extract the TLA+ into a standalone `.tla` plus a `.cfg` so you can actually run TLC on it, or I can write the §4 RPC-bridge hardening as its own short threat-model section (the N-of-M independent-read protocol and how nodes reconcile disagreement). Which is more useful to you right now?



Kms lit integration spec
Document · MD



Download

Ok sounds good, can you update the Stellar integrated health supply plan with all the things discussed? This is purely a requirements level document , don't worry about code/stubs or anything more detailed. It just a synthesis of all things that were discussed, I think we have a good document to start working with for an MVP

Synthesized comprehensive requirements document integrating all discussed decisions >

On it. I'll rewrite the plan as a single synthesized requirements document — folding in the access-broker confirmation, the stable-identity clarification, the consent veto window, the proof-of-presence refinements (active on dispense, preferred-not-required), the collusion fix, the KMS/Lit decision and key recovery, and the second red-team round — all at requirements level, no code or stubs.

Rewriting the plan as a synthesized requirements-level document folding in all discussion ∨

 Rewriting the plan as a synthesized requirements-level document folding in all discussion

Script

 Done

   



